

Bachelor's Degree in Computer Engineering

# AI for Game Development

Final Project

Documentation



Vladyslav Grechyshkin  
Nicolás Fuentes Turpín  
Jose Miguel Sánchez Almagro

---

June 12, 2026

# Contents

|          |                               |          |
|----------|-------------------------------|----------|
| <b>1</b> | <b>Introduction</b>           | <b>1</b> |
| <b>2</b> | <b>Software used</b>          | <b>1</b> |
| <b>3</b> | <b>User interface</b>         | <b>2</b> |
| 3.1      | Laboratory scenes . . . . .   | 2        |
| 3.2      | Final scene . . . . .         | 2        |
| 3.2.1    | User control . . . . .        | 2        |
| 3.2.2    | Graphical interface . . . . . | 3        |
| <b>4</b> | <b>Block I. Movement</b>      | <b>5</b> |
| 4.1      | Class hierarchy . . . . .     | 5        |
| 4.2      | Additional scripts . . . . .  | 5        |
| 4.3      | Basic steerings . . . . .     | 6        |
| 4.3.1    | Proposed scene . . . . .      | 6        |
| 4.3.2    | Steerings . . . . .           | 7        |
| 4.4      | Delegate steerings . . . . .  | 8        |
| 4.4.1    | Proposed scene . . . . .      | 8        |
| 4.4.2    | Steerings . . . . .           | 9        |
| 4.5      | Kinematic steerings . . . . . | 10       |
| 4.5.1    | Proposed scene . . . . .      | 10       |
| 4.5.2    | Steerings . . . . .           | 11       |
| 4.6      | Obstacle detection . . . . .  | 11       |
| 4.6.1    | Proposed scene . . . . .      | 11       |
| 4.6.2    | Steerings . . . . .           | 12       |
| 4.7      | Group steerings . . . . .     | 12       |
| 4.7.1    | Proposed scene . . . . .      | 12       |
| 4.7.2    | Steerings . . . . .           | 13       |

|          |                                                  |           |
|----------|--------------------------------------------------|-----------|
| 4.8      | Formations . . . . .                             | 14        |
| 4.8.1    | Proposed scene . . . . .                         | 14        |
| 4.8.2    | Steerings . . . . .                              | 15        |
| 4.9      | Pathfinding . . . . .                            | 17        |
| 4.9.1    | Proposed scene . . . . .                         | 17        |
| 4.9.2    | Steerings . . . . .                              | 18        |
| 4.10     | Debug mode . . . . .                             | 19        |
| <b>5</b> | <b>Block II. Strategic and tactical AI</b>       | <b>21</b> |
| 5.1      | Unit and terrain types . . . . .                 | 21        |
| 5.1.1    | Units . . . . .                                  | 21        |
| 5.1.2    | Terrains . . . . .                               | 23        |
| 5.1.3    | Individual tactical <i>pathfinding</i> . . . . . | 23        |
| 5.2      | Developed scene . . . . .                        | 24        |
| 5.3      | States . . . . .                                 | 24        |
| 5.3.1    | Set of states . . . . .                          | 25        |
| 5.3.2    | State diagram . . . . .                          | 27        |
| 5.3.3    | Combat modes . . . . .                           | 28        |
| 5.4      | Implemented classes . . . . .                    | 28        |
| 5.4.1    | Graphical interface . . . . .                    | 28        |
| 5.4.2    | Strategy . . . . .                               | 29        |
| 5.4.3    | State machine . . . . .                          | 30        |
| 5.5      | Additional elements . . . . .                    | 30        |

# **1 Introduction**

This document is the report for the final, graded project of the course, following the format, structure and content requirements specified in its guidelines. The project consists of implementing several artificial-intelligence elements in real-time war videogame environments. Following the structure of the practical assignment, these elements are split into two fundamental blocks: movement (reactive AI) and strategy (strategic and tactical AI).

Thus, following the chronological order of the course and the sections indicated in the assignment, the most relevant design and implementation details of these blocks are the following:

## **2 Software used**

This project was developed exclusively with the Unity game engine, using the same version available in the faculty labs: 2019.2.17f1. No platform, library or language other than those used within Unity is of note.

## 3 User interface

Keeping the order and structure of the document, the explanation of the user interface is broken down per block, even though there are many elements common to both.

### 3.1 Laboratory scenes

All the demonstration scenes for the first block's elements use the same interface and user controls, namely:

- **Unit control.** The user can select the units marked as selectable using the left mouse button. This allows selecting a single unit by left-clicking on the desired unit, and multiple selection by left-clicking and dragging a rectangular region that includes the units to be selected. Likewise, elements can be added to the current selection using any of the described methods together with the left **SHIFT** key. Selected units are highlighted with a light-green outline. Once the units of interest are selected, the user can use the right mouse button to indicate a region of the scene to which the selected units must move. These units move using the *steering* behaviours applied to them.
- **Camera control.** The user can move around the scene by panning the camera horizontally and vertically with the **WASD** keys, rotate the camera in increments/decrements of 90° with the **Q** and **E** keys and, finally, zoom the camera in and out with the mouse wheel.

### 3.2 Final scene

All of the block-II elements to be implemented are included in the developed final scene (described in that block's section). Therefore, the user interface specified below refers exclusively to that scene.

#### 3.2.1 User control

The user has exactly the same controls (except rotation) as for the laboratory scenes. Additionally, they can press the **G** key to add units to the same group and thus visualise the group tactical behaviour of the implemented *pathfinding*. Selected units that belong to a group have a yellowish outline. If, among the selected units, some belong to the same group, the remaining un-selected members of that group are highlighted with an orange outline. Note, however, that this does not mean those latter units are selected. Likewise, to remove any selected unit from a group, if it belongs to one, the **C** key can be pressed. Finally, pressing the **M** key toggles the background music on and off. The following restrictions and considerations apply when creating groups:

- A unit can belong to only one group. If a user assigns a unit that already

belongs to a group to another group, that unit stops belonging to the previous one.

- The units of a group must belong to the same team.
- Groups must be made up of at least two units. If a group is reduced to a single member, the group ceases to exist.

It is also worth noting that, unlike in the laboratory scenes, the user can only move a single unit at a time. Therefore, multiple-unit selection is limited exclusively to group creation. Other movement restrictions include right-clicking on unreachable parts of the map or on safe zones.

### 3.2.2 Graphical interface

The graphical elements of the final scene can be classified into two different categories: those associated with a unit and those associated with the game in general. Figure 1 shows these elements running simultaneously.



Figure 1: Example of the final scene running.

Regarding the units, they are made up of the following visual elements:

- **Circular token.** Defines the unit, its team (red or blue) and the type of unit, according to the colour palette and icons used. The winged-shoe icon corresponds to the *scout* unit, the fist to the *heavy*, the plus symbol to the *medic* and the sniper scope to the *sniper*.

- **Health and ammo bars.** Located at the top of the unit, they indicate the unit's current health and ammo relative to the established maximum.
- **Orientation.** Represented by a triangular icon around the circular token described above, it indicates the unit's current orientation. This element is purely aesthetic and does not influence the unit's behaviour.
- **Current state.** Denoted by the grid-like icon at the bottom right of the unit, it shows the unit's current state, corresponding to the states described in the strategic and tactical AI section.
- **Health change.** When a unit is healed or takes a certain amount of damage, it is shown numerically at a random position around the unit. If the amount is healing, the numeric indicator is green; if it is damage, the indicator is red. If the amount received is a critical hit, it is shown as text above the receiving unit.

Regarding the elements associated with the game in general, we distinguish:

- **Mini map.** At the top right of the screen, the influence map used for tactical *pathfinding* is shown. Warm colours correspond to the red team's influence and cool colours correspond to the blue team's influence.
- **Combat modes.** Just to the left of that mini map are the buttons to change each team's combat mode (denoted by colours). These modes are the ones indicated in the practical assignment. Defensive mode corresponds to the buttons whose icon is a shield, and offensive mode to the buttons whose icon is a sword. Likewise, the button whose icon is a banner with spears represents total-war mode. When pressed, the previous modes are deactivated and both teams enter total-war mode.
- **Kill feed.** Located below the mini map, it shows the combat kills generated during the match. These kills are shown for several seconds and imitate the format used in the game taken as a reference for this project (Team Fortress 2). From left to right, it indicates the attacker's name (in their team's colour), the weapon used (a different icon for each class and for melee/ranged attacks), whether it was a critical hit (via a red blur) and the victim's name (in their team's colour). At most four entries are shown at any time, in chronological order.
- **Capture status.** Not visible in the figure referenced so far, both capture points required for each team's victory have a bar similar to the health and ammo bars to indicate the capture progress.
- **Victory.** When a team wins, the winning team is shown in the centre of the screen and the game is paused.

## 4 Block I. Movement

The first block consists of developing a *steering* system in a general scene that, in turn, is made up of multiple individual *steerings*. To build it, the class hierarchy proposed by the teaching staff and the pseudocode from the second edition of the book “Artificial Intelligence for Games” [1] were followed.

Since the pseudocode on which each individual *steering* is developed is extensively explained both in the course theory and in the book itself, only implementation nuances that differ or are significant enough are indicated, provided the corresponding pseudocode was followed.

Likewise, for convenience, the explanation of the implemented *steerings* is organised according to the “laboratory” scene to which they belong. These scenes, unless stated otherwise, contain the following elements:

- **Controls:** The user can move the camera and those units marked as selectable, according to the specification in the “User interface” section.
- **Units:** One or more triangular-shaped units are included for each *steering* contained in the scene. These units are denoted by a clearly distinguishable colour, indicated in this report.

Any other modification to the general structure of the scene is indicated in the corresponding sections.

### 4.1 Class hierarchy

The only differences worth noting with respect to the original hierarchy are the direct inclusion of the **Physical Body** class into the **Agent** class and the addition required for velocity-based steerings. This addition breaks down into three elements: a structure, **KinematicSteering** (velocity and rotation), an additional method in the **SteeringBehaviour** class called **GetKinematicSteering**, and its corresponding application in the **AgentNPC** class, **applyKinematicSteering**. A boolean is used to determine whether the agent is based on velocity steerings or not.

In this way, each individual *steering* can choose to implement its velocity-based version without creating multiple agent and *steering* classes. In the case where this version is not desired or is not conceptually possible, it is enough to return a null *steering* to represent that the method is unimplemented.

### 4.2 Additional scripts

To build the test scenes, a series of auxiliary *scripts* were used, notably:

- **CameraController.** Allows controlling the camera according to the correspond-



ing specification.

- **InputManager**. Manages the user's interaction with the selectable units, imitating the behaviour of a strategy game.
- **DebugManager**. Used for the basic and delegate *steerings*. It automatically assigns the **target** of those *steerings* whose target is a vector but where another agent is wanted instead. In parallel, it initialises the path to be followed by the path-following *steering*.
- **GroupController**. Used for the group *steerings*. It initialises the *steerings* of each unit when the scene runs.

## 4.3 Basic steerings

This section covers all the uniformly accelerated movement *steerings* corresponding to the **Basic Steerings** scene.

### 4.3.1 Proposed scene

The demonstration scene is shown in Figure 2. Note that the textual annotations were added afterwards to aid comprehension and are not part of the scene itself. The configured behaviour of each of the units is as follows:

- The units containing the **Seek** and **Arrive** steerings are selectable and controllable by the user.
- The **VelocityMatch** unit imitates the velocity of the **Seek** unit.
- The **Align** unit imitates the orientation of the **Seek** unit, and **AntiAlign** does likewise but with the opposite orientation. To this end, a **Look Where You're Going** steering is added to the **Seek** unit so that its orientation can be modified, even though it is not a basic *steering*.
- The **Flee** unit flees from the **Seek** unit if it is within an arbitrarily established radius.



Figure 2: Elements available in the Basic Steerings scene.

#### 4.3.2 Steerings

##### Seek

In the scene, this *steering* is associated with the light-green unit. The only difference with respect to the pseudocode is the insertion of a **threshold** variable to avoid the sliding or drifting inherent to this type of *steering*. If the agent is at a distance smaller than **threshold**, a negative acceleration equivalent to the agent's current one is applied.

##### Flee

This *steering* is associated with the orange unit. Analogously to its opposite *steering*, Seek, it also relies on a **threshold** variable to avoid the unit drifting.

##### Arrive

This *steering* has the dark-green unit associated with it. For this *steering*, the meaning of returning a “null” *steering* when the unit has reached its destination is left to free interpretation. The chosen interpretation is the same as for the previous *steerings*: when the agent has reached its target, negatively apply the agent's current acceleration.

##### Align

The unit associated with this *steering* is blue. Conceptually, there is no difference with respect to the pseudocode, although the conversion to the interval  $(-\pi, \pi)$  is explicit and not done via an external function.

## Anti-Align

This *steering* is applied to the red unit. To obtain the orientation opposite to **Align**, it is enough to add  $\pi$  to the computed rotation; the rest of the code is identical to that of **Align**.

## Velocity Matching

The white unit represents this *steering*. No improvement or change was made to the original pseudocode for its implementation.

## 4.4 Delegate steerings

Similarly, this section includes those *steerings* based (class inheritance) on the previous basic ones, corresponding to the **Delegate Steerings** scene.

### 4.4.1 Proposed scene

The configured behaviour for this scene (see Figure 3) is as follows:

- There is no unit selectable and controllable by the user.
- The **Pursue** unit chases, at a distance of two units, the **Wander** unit, while the latter moves freely around the scene.
- The **PathFollowing** unit “patrols” the path indicated by the white cubes.
- The **Face** unit orients itself looking at the **PathFollowing** unit.
- The **Pursue**, **PathFollowing** and **Wander** units orient themselves in the direction they move, using the **LookWhereYoureGoing** *steering*.

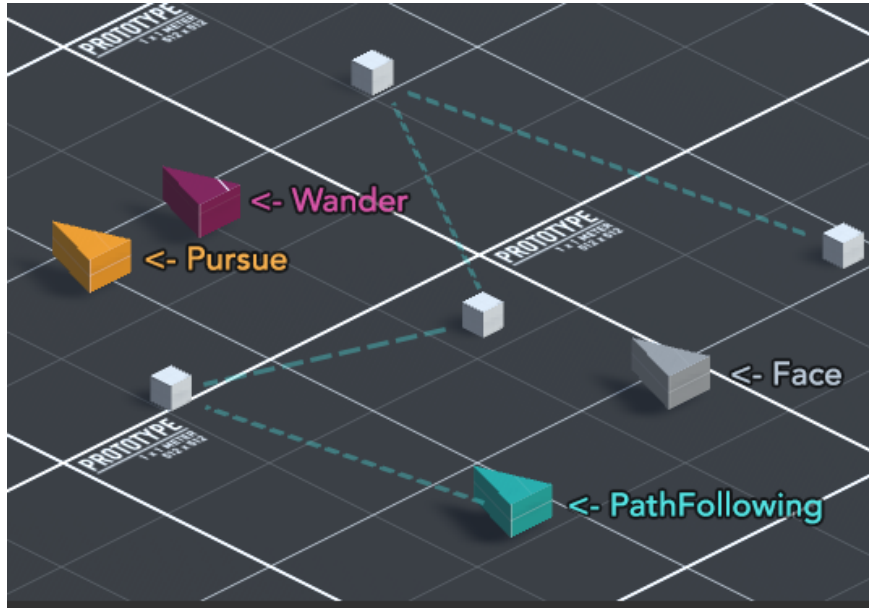


Figure 3: Contents of the Delegate Steerings scene.

#### 4.4.2 Steerings

##### Pursue

The orange unit represents this *steering*. No improvement or change was made to the original pseudocode for its implementation.

##### Face

This *steering* is applied to the grey unit. In this implementation, only a check was added to change the orientation only if we have a **Target** to look at.

##### Look Where You're Going

This *steering* is associated with the Wander, PathFollowing and Pursue units. We interpret the pseudocode's **return** call as a **return none**, as in **Arrive**. This is achieved by returning the current orientation as the target one.

##### Wander

In the scene, this *steering* is associated with the wine-coloured unit. Again, for this *steering* we chose to follow the proposed pseudocode (inheriting from the **Face** class). There is no modification or significant contribution to the implemented code.

##### Path Following

The unit associated with this *steering* is turquoise. Following the structure suggested in the book, this *steering* is composed of two complementary classes: **PathFollowing**

and `Path`.

The first presents no difference with respect to the pseudocode. By comparison, for the second only an interface is provided, and the programmer must choose their own strategy to determine the agent's position along the path to follow. Thus, the chosen implementation is based on line segments. The specific details of each method described in the interface are:

- **GetParam**: Returns an integer that represents the agent's current point on the path as a function of a previous point passed as a parameter. If the previous position is outside the limits, the returned position is the corresponding limit. Otherwise, the criterion used to obtain the current point is the following: if the agent is closer (Euclidean distance) to the previous or next point and within an established radius, that point is returned. Otherwise, the current point is returned.
- **GetPosition**: Returns the scene position associated with the integer obtained in **GetParam**.
- **AppendPointToPath**: Adds a point at the end of the current path.
- **ClearPath**: Removes all the points of the path and initialises the path to the current position of the object associated with this *steering*.

## 4.5 Kinematic steerings

This section gathers all the kinematic *steerings* corresponding to the **Kinematic Steerings** scene.

### 4.5.1 Proposed scene

The configured behaviour for this scene (see Figure 4) is as follows:

- The unit containing the **Seek** steering is selectable and controllable by the user.
- The **Wander** unit moves freely around the scene.
- The **PathFollowing** unit “patrols” the path indicated by the white cubes.
- The **Flee** unit flees from the **Seek** unit if it is closer than an arbitrarily established distance.

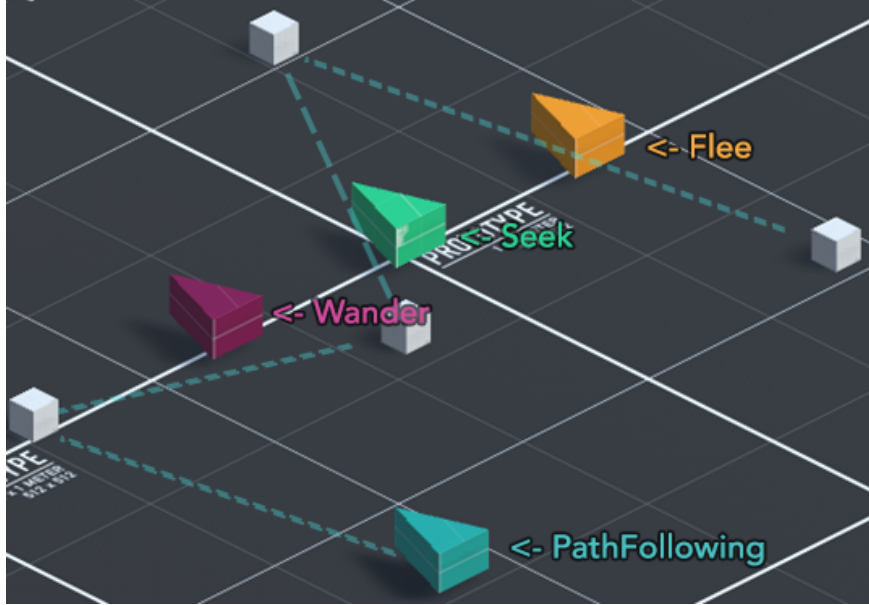


Figure 4: Contents of the Kinematic Steerings scene.

#### 4.5.2 Steerings

The code implemented in these kinematic *steerings* is analogous to their acceleration-based versions in the case of **Seek**, **Flee** and **PathFollowing**. In the case of **Wander**, the corresponding pseudocode is followed.

### 4.6 Obstacle detection

This section includes the remaining delegate *steering*: **ObstacleAvoidance**. The main and only reason for separating this *steering* from the rest of the delegate *steerings* is to reduce the content load of each demonstration scene.

#### 4.6.1 Proposed scene

The layout and configuration of the elements of the built scene (Figure 5) is:

- A single unit, selectable and controllable by the user, with **Seek**, **ObstacleAvoidance** and **BlendedSteering**.
- Multiple orange obstacles to visualise the operation of the **ObstacleAvoidance** *steering*.

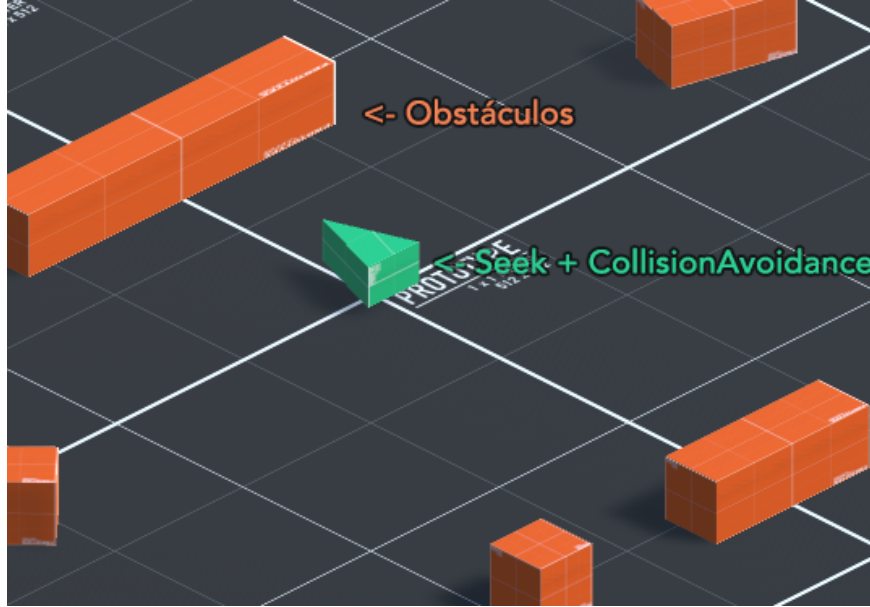


Figure 5: Contents of the `Obstacle Avoidance` scene.

#### 4.6.2 Steerings

Note that, although the demonstration of `ObstacleAvoidance` uses two additional *steerings*, these are explained in their respective sections.

#### Obstacle Avoidance

Associated with the light-green unit, the strategy followed for the implementation is the use of the normals obtained from three collision rays: one main frontal ray and two smaller auxiliary ones positioned at a modifiable angle with respect to the main one.

Thus, the procedure described in the pseudocode is repeated for each of the collision rays. Otherwise, the code is the same.

### 4.7 Group steerings

The *steerings* described in this section correspond to those seen in the course that are needed to incorporate *flocking* behaviour.

#### 4.7.1 Proposed scene

The elements available in the scene (Figure 6) and the configuration of their behaviour are as follows:

- There is no unit selectable and controllable by the user.

- Three groups of three units with **Alignment**, **Separation** and **Cohesion** behaviour respectively.
- A group of twelve units with combined **Alignment**, **Separation** and **Cohesion** behaviour, via **BlendedSteering**, to achieve a *flocking* effect.

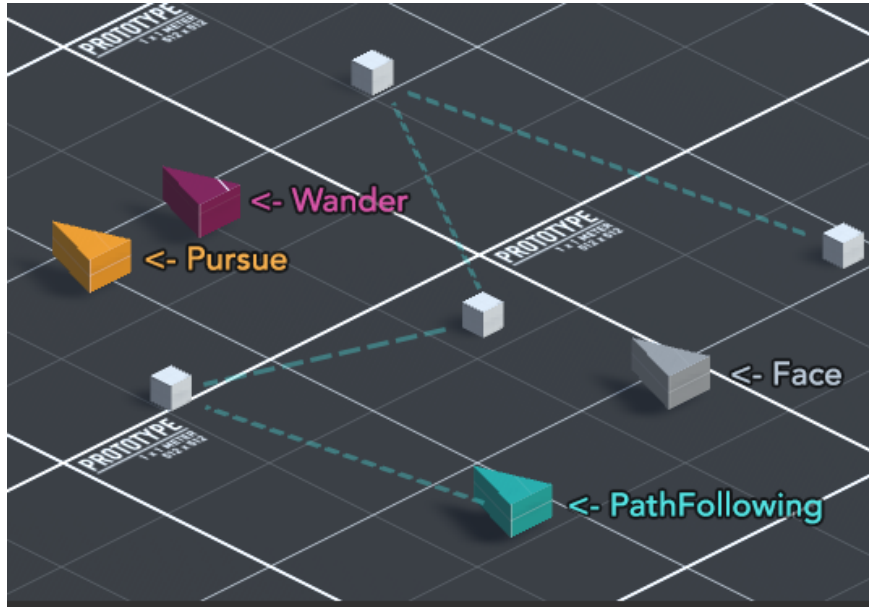


Figure 6: Group Steerings.

#### 4.7.2 Steerings

##### Blended Steering, Separation and Cohesion

There are no significant contributions with respect to the available pseudocode.

##### Alignment

This *steering* derives from the **Align** class and, following the pseudocode, the *steering*'s behaviour does not match the desired one. This is because there is a bug in the original pseudocode: the instruction `Heading -= character.Heading()` makes no sense; it bears no relation to the average computed before that instruction.

##### Flocking

*Flocking* brings together four already-discussed *steerings*: **Wander**, **Separation**, **Cohesion** and **Alignment**, via a weighted blend applied with **BlendedSteering**.

Several modifications were also made here with respect to the pseudocode from the slides. The acceleration limits that in the pseudocode are done with the maximum function are replaced. The minimum function is used instead, since it is somewhat incoherent to limit a value by choosing the maximum of two. On the other hand, for



the `linear` attribute of the *steering* the minimum is not simply taken; that obtained value is multiplied by the normalised vector to obtain the desired acceleration.

## 4.8 Formations

This section describes the classes designed to guarantee the generation of fixed- and scalable-structure formations at two levels.

### 4.8.1 Proposed scene

Since it is only necessary to demonstrate the transition from a disordered state to an “ordered” state, all the developed formations are presented in two scenes (Figure 7), separated according to whether they are fixed or scalable formations. The full description of the elements integrated into the scenes is as follows:

- There is no unit selectable and controllable by the user in the scalable-formations scene. For the fixed-formations scene, the user can control the leaders of each formation (indicated by a yellow circle in the corresponding figure) and, with the Z key, make the other members of the formation go to their respective positions.
- A total of seven groups of units are included, each identified by a different colour. Two of the groups have a different fixed-structure formation associated with them, namely: `FingerPattern` and `BoxPattern`. The remaining four groups have scalable formations; one group is arranged with a `LinePattern` formation and the other three with `DefensiveCirclePattern` (with different numbers of units).
- When the scenes are initialised, the unit groups transition to their “ordered” state based on the corresponding leader or the computed centre of mass.



(a) Fixed Formations



(b) Scalable Formations

Figure 7: Scenes demonstrating the behaviour of the developed formations.

## 4.8.2 Steerings

In themselves, the formation structures do not present a new *steering*; rather, they use previous *steerings* to generate a fixed or emergent structure. The classes used to represent these structures are similar but with notable differences. The diagram in Figure 8 shows the class hierarchy used.

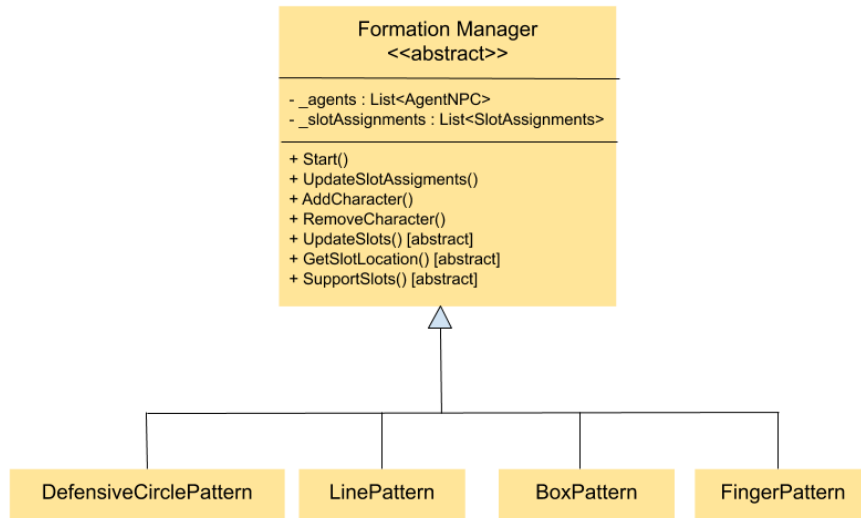


Figure 8: Class diagram for the formations.

In this hierarchy, there is an abstract class called **FormationManager** from which all the different concrete formations inherit. Likewise, within the abstract class, two different structures are used to synthesise the handling of the variables involved: **Location**, representing a given position and orientation of a formation member, and **SlotAssignment**, representing the agent and its position within the formation. As for the functionality, on the one hand, the methods that each subclass must implement to generate the desired formation are:

- **UpdateSlots**. Can be considered the core of the class. Its function is to assign a position and an orientation to each element that makes up the formation.
- **GetSlotLocation**. Given a slot number that represents a formation element, returns its respective **Location**.
- **SupportSlots**. Returns a boolean indicating whether a certain number of units is allowed in the formation. For scalable formations it is always *true*.

On the other hand, the remaining methods that use the abstract methods are:

- **UpdateSlotAssignments**. Assigns a numeric value to each position or *slot* of the formation.

- **AddCharacter.** Adds an agent to the formation (based on **SupportSlots**) and updates the values of each position via **UpdateSlotAssignments**.
- **RemoveCharacter.** Removes an agent from the formation (if it belongs to it) and updates the values of each position via **UpdateSlotAssignments**.

### Two abreast in cover

Programmed in the **BoxPattern** class and represented by the blue units, it is a fixed formation of four units in a defensive “box” shape. For its implementation, a leader is used (obtained via the auxiliary function **GetLeader**); all formation positions are relative to the leader. Once the leader and all the corresponding **Locations** are obtained, the formation members are iterated over and each member’s target position and rotation are assigned.

### Finger four

Implemented in the **FingerPattern** class and integrated into the dark-green units, its behaviour is the same as that of the previous formation. The only difference, obviously, is the specific position of each formation member generated by **GetSlotLocation**. The final result is a formation shaped like the fingers of a hand (excluding the thumb).

### Line

Available in the **LinePattern** class, it is the first of the two scalable formations programmed (it has no maximum number of units that can be assigned to it). In the scene it corresponds to the yellow units. It produces a formation in which all members are positioned in parallel along a line, facing the same direction. Unlike the previous formations, it does not use a leader but a two-level structure; the anchor point used is the average position and orientation of the formation members (obtained via **GetAnchor**). Note, however, that the orientation could be parameterised if the line is to face a specific direction.

To obtain the positions of each formation member (that is, the **GetSlotLocation** function), a simple arithmetic position-offset operation is performed.

### Defensive circle

Finally, the last implemented formation is in the **DefensiveCirclePattern** class. For its development, the pseudocode available in the bibliography used was followed, except for the *drift*. The only difference from the original code is taking the members’ position in the scene into account to obtain the anchor point. In the scene where it is included, the units (grouped by colour) with this formation type are the white, orange, light-green and wine-coloured ones.

## 4.9 Pathfinding

This section includes the last developed *steering*: path search, or *pathfinding*. As specified in the practical assignment, the specific variants to implement are LRTA\* (mandatory for the first block) and A\* (optional for the second block).

### 4.9.1 Proposed scene

Two different scenes are distinguished, one for each variant. The first scene (Figure 9), described in this section, serves to demonstrate the operation of LRTA\* and its contents are described below. Conversely, since the use of A\* would be reflected in the second block, its demonstration scene would be the general scene to be developed in that block. However, although the initial intention was to implement A\*, and the proposed class hierarchy accommodates both variants, the final decision was to use only LRTA\*.

Thus, the contents are:

- Two units with **Pathfinding**, selectable and controllable by the user, one kinematic and the other acceleration-based. By default, only the kinematic unit is enabled, since it visualises the obtained path without depending on the drifts generated by accelerations. However, if the user wishes, they can use the other unit by enabling the **Acceleration Pathfinder** object from the scene.
- A scene divided into square cells with obstacles, through which the unit can move. The scene limits are denoted by the yellow tint applied to the visitable cells. The objective of this scene is to visualise the “vision” limitations of LRTA\*.



Figure 9: Contents of the Pathfinding scene.

### 4.9.2 Steerings

For the implementation of this *steering*, a class structure completely unrelated to the one referenced in the bibliography used so far was followed. We highlight two fundamental elements for the operation of this *steering*: the square map (*grid*) and the *pathfinding*.

#### Grid

The structure and operation of the square map is based on the tutorials of the Sharp Accent channel [2]. The specific classes implemented are **Tile** and **GridMap**.

The first, **Tile**, represents an individual cell and its associated information, that is, the cell's position in the grid, global position, the cell's terrain type and whether it is a cell a unit can pass over.

The second, **GridMap**, represents the complete grid made up of individual **Tiles** and allows obtaining any cell, its neighbours, and converting cell positions to scene positions and vice versa. Two methods are used to create the grid:

- **ReadLevel**. Determines the limits and size of the grid based on two markers positioned in the scene.
- **CreateGrid**. Called at the end of **ReadLevel**, it generates a grid of a given size, initialising each individual cell. For each cell, the terrain type and obstacles are checked by looking at the objects residing within the cell. If an obstacle is found, the cell is marked as impassable. Otherwise, the corresponding terrain type is assigned to it.

#### Pathfinding

By a completely subjective decision, path search was implemented on top of the path following described earlier. The resulting hierarchy breaks down into three classes: **PathFinding**, the abstract super-class from which the following two inherit; **LRTA** and **A** (not implemented).

**PathFinding** inherits from the **PathFollowing** class and includes the information common to both concrete *pathfinding* implementations developed. This information is the computation of the heuristics from one cell to another based on different types of distances (Manhattan, Chebyshev and Euclidean). The construction of each of the distance-computation methods is based on the explanations by Amit Patel [3].

Then, **LRTA** and **A** (had it been implemented) inherit from the **PathFinding** class and implement the abstract method **FindPathToPosition** which, given a start and end cell, searches for and generates the path from one end to the other. This method is used by the super-class as a coroutine to avoid sequential computation by the agent.

The implementation of the LRTA\* algorithm was done strictly and faithfully to the bibliography from which the pseudocode developed in the course slides comes [4]. Therefore, the specific structure and methods used correspond directly to the original pseudocode. Nevertheless, the following implementation details, used to improve the algorithm's efficiency and memory, are worth highlighting:

- The implemented algorithm allows any local search space and is not limited to the `LookAheadOne` version, thereby fulfilling one of the proposed optional objectives. The specific search spaces developed are `LookAheadOne` and a breadth-first search (`BreadthFirstSearch`). This breadth-first search adds cells until the established size is reached, in the order characteristic of this type of search: first all the neighbours of the current cell are added, then the neighbours of the neighbours, in the order they were added.
- The use of an internal structure called `TileInfo` to store heuristic information related to the algorithm, plus a lazy copy of the original *grid* with the added information. This approach guarantees the concurrency of *pathfinding* agents by removing critical sections, since each agent stores its heuristic values individually. Similarly, since it is a lazy copy of the square map, each cell's additional information is only initialised once the agent visits it.
- The use of timestamps to reduce the algorithm's initialisation time between one run and another. When the agent finishes its run and starts a completely different one, the updated heuristic values of each cell are no longer valid. Instead of resetting all the heuristic values (which entails a cost equivalent to the number of map cells visited so far), each `TileInfo` is updated when the timestamp at which the algorithm was run does not match the one stored by that `TileInfo`.
- Since the algorithm does not need to know the internal structure of the *grid* when requesting the adjacent cells or “neighbours” via the `GetAdjacentTiles` method of the `GridMap` class, in theory any *grid* could be used, whether square or not. Likewise, the searches for the minimum-cost neighbours depend on the order in which these neighbours are received. In the particular case of the implemented map, it is clockwise, starting with the left neighbour. Any tie that occurs is resolved according to that order.
- Finally, since both this algorithm and A\* only generate the path and do not dictate how it must be traversed, any possible improvement such as *string pulling* can be added. Likewise, by inheriting from the `PathFollowing` class, the agent can move via kinematic rather than acceleration-based movements.

## 4.10 Debug mode

The various *Gizmos* shown in debug mode are detailed below. Everything explained in this section can be enabled from the Unity inspector via a boolean:

- **Agent.** All agents can display their inner and outer radii (used in **Arrive**) as well as their inner and outer angles (used in **Align** and **AntiAlign**). They can also display their velocity vector.
- **AgentNPC.** This agent type can additionally display its acceleration vector (provided the applied *steering* is not kinematic).
- **ObstacleAvoidance.** In this *steering*, the distance our unit keeps from obstacles (in the form of a sphere) and the vectors that check for a possible collision (both the main one and its whiskers) can be displayed.
- **Path.** The **Path** class can display the arrival radius of all the path points, that is, the minimum distance our unit must be at for it to be considered to have reached that point.
- **Pathfinding.** All the cells visited by the unit and their heuristics (as they are updated) can be displayed.
- **GridMap.** All existing cells can be visualised, whether passable (grey) or not (red). For the second block, impassable cells that nevertheless allow visibility (blue) are also distinguished.

## 5 Block II. Strategic and tactical AI

The second and final block consists of developing artificial-intelligence elements in a real-time war game environment. Following the aspects and considerations of the practical assignment, a first-person multiplayer shooter was taken as a reference for its development: Team Fortress 2. All the unit types and their static information are based directly on this game. Likewise, the icons for each unit and other visual elements are also taken directly from it or, failing that, inspired by it.

Except for the mandatory elements specified in the assignment guidelines, the format of the game's structure and logic is entirely subjective, according to the group members' judgement.

### 5.1 Unit and terrain types

The developed environment has four different unit types and three terrain types that influence their tactical *pathfinding*.

#### 5.1.1 Units

**Scout.** This unit type has the highest movement speed of all. Its ranged weapon is a shotgun that fires several low-damage pellets per shot, and its melee-attack and capture speed is twice that of the other units.

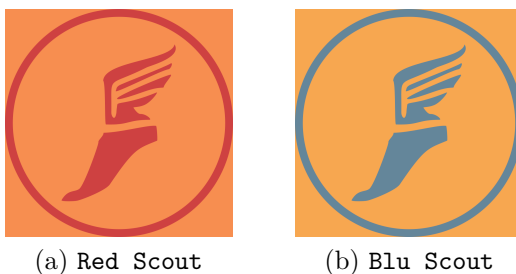


Figure 10: Scout unit icon



**Heavy.** This unit type has reduced movement speed but a large amount of base health. Its ranged weapon is a machine gun with a high rate of fire and a large magazine. This unit type is the only one that patrols during the match.



Figure 11: Heavy unit icon

**Medic.** This is the only unit type able to heal. It has slightly increased health, and its ranged weapon fires darts that wound enemies and heal allies, but it has a very small magazine.

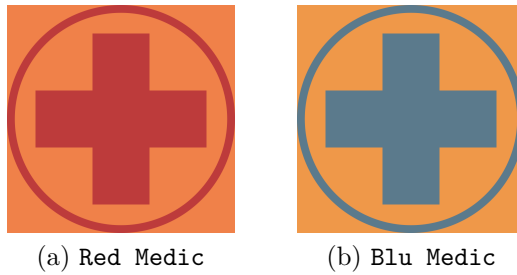


Figure 12: Medic unit icon

**Sniper.** Its main feature is its ranged weapon, a long-range sniper rifle with a long charge time but explosive damage.

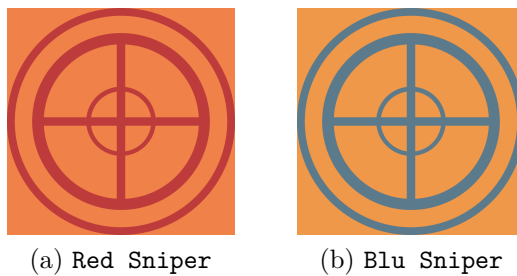


Figure 13: Sniper unit icon

### 5.1.2 Terrains

The following table shows the multipliers applied to each unit's base speed depending on the terrain type of the cell it is on.

| Unit                                                                              | Forest | Grassland | Road |
|-----------------------------------------------------------------------------------|--------|-----------|------|
|  | 0.9    | 1         | 1.25 |
|  | 0.75   | 1         | 1.5  |
|  | 1      | 1         | 1.5  |
|  | 1      | 1.1       | 1.25 |

### 5.1.3 Individual tactical *pathfinding*

To guarantee tactical behaviour in the developed *pathfinding*, modifications to the cost calculation of each cell are included. Thus, a variable is added indicating whether tactical *pathfinding* is applied or not. If it is enabled, the costs associated with terrain, the influence map [5] and visibility are applied. Terrain and visibility costs are added directly to the final cost, whereas influence costs depend on the team the unit creating the path belongs to. If it belongs to the red team, it ignores positive influences and tries to avoid the negative ones; if it belongs to the blue team, it does so the other way around.

## 5.2 Developed scene

The scene developed for the second block is the following:

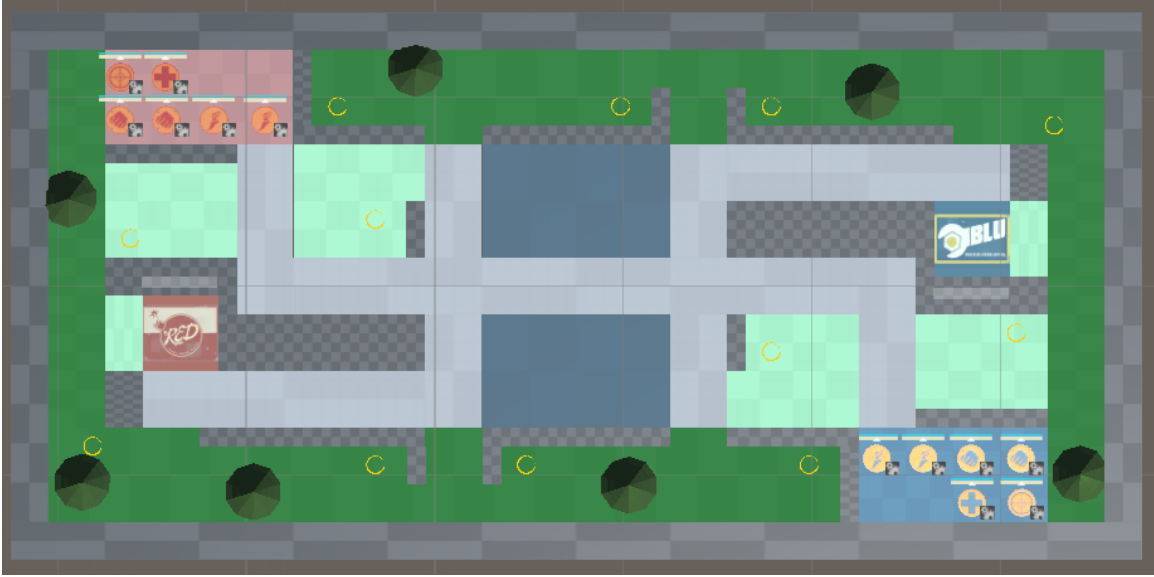


Figure 14: Contents of the **Strategy Scene** scene.

This scene is made up of the different terrain types described in the previous section, together with cover *waypoints* (the yellowish circles), both teams' bases and capture points, obstacles and decorative elements. Breaking it down by colour, the light-grey parts of the map correspond to roads (*Road* in the code), the light-green parts to grasslands (*Grassland*), the dark-green parts to forests (*Forest*), the blue parts to water (impassable zones that nevertheless allow visibility) and, finally, the greyish grid-shaped parts correspond to obstacles (impassable and not allowing visibility).

As for the bases, they are in the colour corresponding to each team (red and blue) and are located in the top-left and bottom-right corners of the scene. Likewise, the capture points are the parts of the map containing each team's logo.

Due to the layout of the scene's elements, the map is divided into two countries (as indicated in the assignment specification), leaving only three different passes between the two regions of the map. This layout forms a rich scene that helps the units operate more strategically and intelligently, and ensures that every match is different, due to the variety of possible situations that may arise.

## 5.3 States

The functionality and behaviour of the units is modelled with a state machine. This machine is composed of a fixed number of states and connections defined to guarantee the units' tactical and strategic behaviour.

### 5.3.1 Set of states

| Icon                                                                                | State   | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|-------------------------------------------------------------------------------------|---------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|    | Idle    | It is the starting state of all units, from which a transition to any other can be made. When the character is in this state it performs no action; it only determines which state to move to. Therefore, all units are only intermittently in this state, to decide which action to take.                                                                                                                                                                                                                                                                                                                                                         |
|    | Dead    | This is the state units go to when they die. While in it they perform no action; they simply wait a preset amount of time before being able to revive. When they revive, they recover all their health and ammo.                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|    | Escape  | When a unit has low health or is in danger it moves to this state. Once in it, it decides whether to escape to base, to an ally (if it is in danger) or to a medic (if it has low health and the medic can heal it) depending on its distance to these three options. If it chooses to flee towards an ally, when the unit reaches that position it checks whether the ally is still there. If not, it flees to base (recovering all health and ammo). If total-war mode is enabled, units never flee.                                                                                                                                             |
|  | Capture | The goal of the game is to capture the enemy zone. Therefore, when a unit meets the right conditions, it enters this state and tries to advance to the enemy checkpoint and capture it, in order to win the match. The entry conditions are that the unit must not be required to patrol (ignored in total-war mode), there are no enemies nearby, there are enough allies capturing (dependent on each unit and combat mode) and it has enough health to do so. Likewise, if an enemy unit reaches the allied capture point, the unit will leave this state if its distance to the allied capture zone is smaller than to the enemy capture zone. |
|  | Defend  | Conversely to the previous state, when an enemy unit reaches the allied capture point, the unit will go to defend it, as long as it is not already capturing the enemy point and has enough health (again, the latter is ignored if total-war mode is enabled).                                                                                                                                                                                                                                                                                                                                                                                    |

|                                                                                     |              |                                                                                                                                                                                                                                                                                                                                                                                                                             |
|-------------------------------------------------------------------------------------|--------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|    | MeleeAttack  | Enables a unit's melee combat. Any unit moves to this state whenever it cannot attack at range (due to having no ammo). To perform a melee attack, it first tries to approach (once) its target. On arrival, it proceeds to attack its target repeatedly until killing it or losing too much health (ignored in total-war mode).                                                                                            |
|    | RangedAttack | Enables a unit's ranged combat. Any unit moves to this state whenever it sees an enemy in its field of view and has ammo. On entering the state, consecutive shots are fired at the target as long as it remains in its field of view and the unit has enough ammo and health (again, the latter is ignored in total-war mode). In the particular case of medics, they can use their ammo to "shoot" an ally and heal them. |
|    | Reload       | When a unit runs out of ammo, it must reload its weapon. It will do so whenever it has no more important action to perform and there are no enemies nearby. On entering the state, it looks for the nearest cover point to go and reload. If, on the way to the cover point, it encounters enemies, it returns directly to base to reload.                                                                                  |
|  | User         | As indicated in the user interface, all units are selectable and movable. When that action occurs, the unit moves directly to this state, in which it only obeys the player's orders. When the unit reaches its destination it automatically leaves this state.                                                                                                                                                             |
|  | Patrol       | Any unit designated to patrol will do so whenever it does not need to perform any of the previous actions. Patrol mode consists simply of moving linearly between two preset points.                                                                                                                                                                                                                                        |
|  | Roam         | If a unit meets none of the conditions of the previous states, it chooses to wander around points of interest in its "country".                                                                                                                                                                                                                                                                                             |
|  | Heal         | Any unit remains in this state while it is being healed, whether by a medic or when it is at base (healing automatically). The unit leaves the state when it recovers enough health (if healed by a medic) or when it fully recovers its health at base. In the latter case, it also recovers its ammo.                                                                                                                     |

### 5.3.2 State diagram

Thus, the described states and their conditions are reflected in the following state diagram:

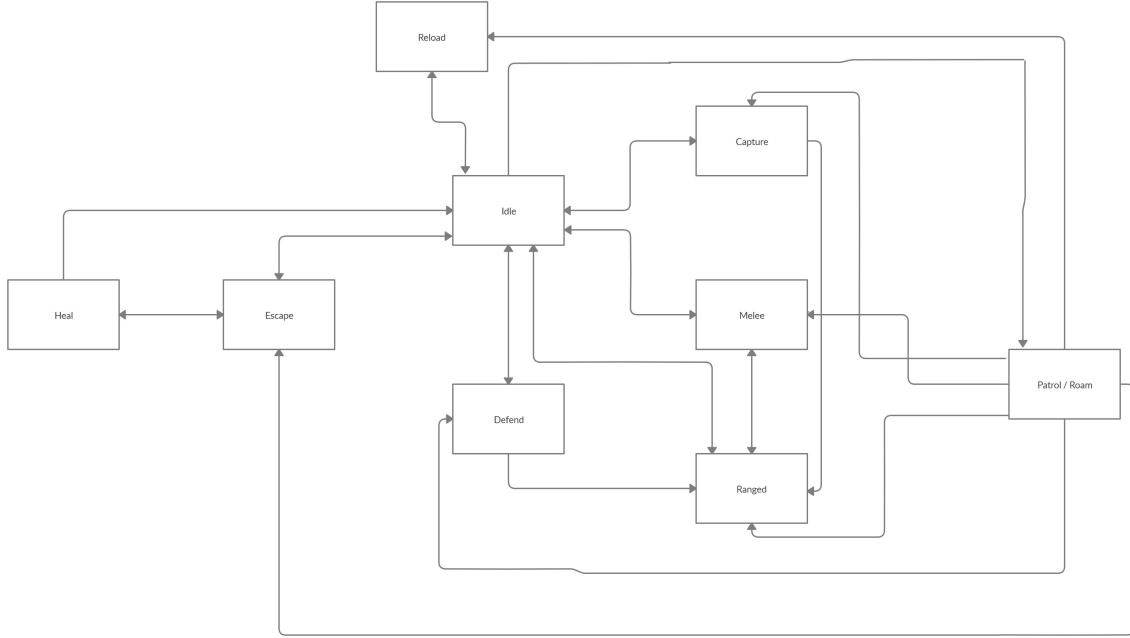


Figure 15: State diagram of the game's units.

Note that two states are not included in the diagram for simplicity:

- **Dead**. All states can transition to **Dead**, so including it would only hinder the diagram's readability. When a unit revives, it can only transition to **Idle**.
- **User**. Similarly to the previous state, all states can transition to **User**, and from it one can only return to **Idle**.

Looking at the diagram, the notion that **Idle** acts as a central state from which all other actions can occur is solidified. However, this does not mean that the only way to reach a given state is through **Idle**. Similarly, the **Patrol** and **Roam** states behave alike. Since both states are only accessed if the other actions are not possible, the units' flow is simplified by letting these states check all the others. Finally, it is worth noting that **Heal** is only accessible from **Escape**, since in all cases, in order to heal, the unit must flee.

### 5.3.3 Combat modes

These transitions between states can vary depending on the combat mode active during the match. There are 3 different modes, detailed below.

| Icon                                                                                | Mode           | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|-------------------------------------------------------------------------------------|----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|    | Defensive Mode | It is the default mode. While active, the defence of the allied capture point is prioritised by lowering the maximum number of enemies to flee from and to melee-attack, increasing the importance of enemy influence in the <i>pathfinding</i> , raising the minimum number of allies required to melee-attack and reducing the units' <b>LowHealth</b> and <b>Healthy</b> values. Additionally, in this state units decide to go and capture if they have a certain number of allies at the enemy capture point, except for the <b>Scout</b> . |
|   | Offensive Mode | This mode prioritises conquering the enemy capture point. This is achieved by increasing the maximum number of enemies that must be nearby for a unit to decide to flee or melee-attack. Likewise, the number of allies needed to melee-attack and the <i>LowHealth</i> and <i>Healthy</i> stats are decreased. In this mode all units are able to capture without an ally restriction, and the importance of the visibility map relative to the influence map is increased in the <i>pathfinding</i> .                                          |
|  | Total War Mode | In this mode, “bravery” prevails over any objective, completely removing the escape mechanic. Likewise, units that must patrol stop doing so. The <b>LowHealth</b> and <b>Healthy</b> values decrease significantly, as does the minimum number of allies required to engage in melee combat. In this mode, tactical <b>pathfinding</b> does not take enemy influence or visibility into account, and all units can capture without restrictions. The desired effect is to seek the victory conditions at any cost.                              |

## 5.4 Implemented classes

Three sets of classes necessary for the operation of the second block's elements stand out.

### 5.4.1 Graphical interface

This set encompasses all the classes related to the operation of the graphical interface, namely:

- **GUIManager**. Handles all the calls related to updating the scene's visual elements (animations, state changes, image loading, etc.).
- **GUIKill**. Constitutes an individual kill entry, responsible for initialising itself with the appropriate values according to the kill that occurred during combat.
- **GUIKillFeed**. Composes the complete kill feed, managing the generation and updating of multiple **GUIKill**.
- **GUICaller**. Used strictly to make event calls during the state-change animation.
- **FloatingText**. Responsible for initialising and destroying the numeric indicators generated during health changes.

### 5.4.2 Strategy

This second set corresponds to the classes used to control the game state, user input and interactions between units. These classes are:

- **CombatManager**. Acting as a facade pattern for handling the game's combat, it provides the methods needed to perform the different types of available attacks (melee and ranged).
- **StrategyInfluenceManager**. Fundamentally, it is an extended copy of **InputManager**. In order not to overload a class already used for the laboratory scenes, this class adopts all the previous functionality and adds the group creation described in the user interface.
- **GroupManager**. Static auxiliary class used by **StrategyInfluenceManager** to correctly keep track of all existing groups. Alternatively, it could be included directly within that class.
- **GameManager**. Constitutes the main core of the game flow. Attack-mode changes and the retrieval of tactical information (attacking enemies, defending allies, etc.) are done through this class.
- **UnitsManager**. Static auxiliary class for computing tactical information strictly related to the units' positions, such as the number of nearby enemies or allies of a unit.
- **Waypoint**. Stores the static information of the different *waypoints* available in the game. In the case of capture points, the corresponding visual indicator is also updated.
- **WaypointManager**. As its name indicates, it handles retrieving information about the available *waypoints*.



### 5.4.3 State machine

The last set includes all the classes related to the individual logic of each unit participating in the scene. These classes are:

- **NPC.** Contains all the static and dynamic information related to each unit. It also controls the execution of the implemented state machine and its visual elements.
- **State.** Abstract super-class that defines all the information and logic of a state, as well as the checks needed to transition from one state to another.
- Finally, the classes for each of the states covered in the previous sections. Their names and behaviour are as already described.

## 5.5 Additional elements

To meet the requirements for the assignment’s evaluation in the case of a three-person group, the following additional elements (from those specified in the assignment guidelines) were chosen:

- Group-level tactical *pathfinding*. Making use of the group creation specified in the user-interface section, the tactical *pathfinding* used by each unit can be modified. Since the way groups should influence *pathfinding* is not specified, the chosen approach is only to modify the speed and terrain costs of the units belonging to a group so that they all share the values of the slowest unit. That is, if a group is formed between a *heavy* and a *scout*, the scout will move at the *heavy*’s speed and have its terrain costs. Optionally, the weights used for tactical behaviour could also be modified based on the “worst” unit.
- Visibility. A visibility map was implemented and is used in the tactical *pathfinding* costs. This visibility map is generated at the start of the game and, for each cell, stores, approximately, the number of cells visible from that cell. Note that the value stored in each cell of the visibility map is normalised and inverted so that cells with a “smaller” visibility value are the most favourable. The reason for this modification is to simplify and keep uniform the calculations performed by the tactical *pathfinding*.
- Mixing levels and other tactical-information properties. Again, this part is highly generic and subjective, so there are multiple approaches for the same project. The additional information implemented is: the use of cover points to reload ammo, searching for medic units to receive healing if they are closer than the allied base, and the consideration of danger in decision-making (in the form of the number of nearby enemies and allies, to decide whether to flee or attack).

## References

- [1] Ian Millington and John Funge. *Artificial Intelligence for Games*, Second Edition. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 2nd edition, 2009.
- [2] Sharp Accent. *Turn Based Game Part 1 — Grid Creation, Unity Tutorial (Advanced)*. YouTube, 2019. <https://www.youtube.com/watch?v=x7drd0tSftk>
- [3] Amit Patel. *Heuristics — Amit's Thoughts on Pathfinding*, 1997. <https://theory.stanford.edu/~amitp/GameProgramming/Heuristics.html>
- [4] Stefan Edelkamp and Stefan Schroedl. *Heuristic Search: Theory and Applications*. Elsevier, 2011.
- [5] Ferdinand Joseph Fernandez. *Influence Maps in Unity 3D*, 2012. <https://bitbucket.org/AnomalousUnderdog/influencemapsunity3d/src/Dev-Main/InfluenceMapUnity/>